

Sicherheitsaspekte von Webassembly

Projektarbeit

David Weber

3015004

15.07.2026

Betreuer:

Prof. Dr. Christoph Karg



Hochschule Aalen
UNIVERSITY OF APPLIED SCIENCES

Hochschule Aalen
Studiengang Informatik

Inhaltsverzeichnis

1	Einleitung	2
1.1	Motivation	2
1.2	Problemstellung	2
1.3	Zielsetzung	3
1.4	Forschungsfragen	3
1.5	Aufbau der Arbeit	3
2	Grundlagen	5
2.1	WebAssembly	5
2.1.1	Einordnung, Entstehung und Standardisierung	5
2.1.2	Designziele von WebAssembly	5
2.1.3	Ökosystem und Einsatzbereiche	6
2.2	Architektur von WebAssembly	6
2.2.1	Modulkonzept	6
2.2.2	Ausführungsmodell	6
2.2.3	Speichermodell und Isolation	7
2.2.4	Typisierung und Validierung	7
2.2.5	Sandbox-Modell	7
2.2.6	Host-Integration und Laufzeitumgebungen	7
2.3	Sicherheitskonzepte im Web	7
2.3.1	Same-Origin Policy	7
2.3.2	Content Security Policy	8
2.3.3	JavaScript-Sicherheitsmodell	8
3	Methodik	9
3.1	Literaturrecherche	9
3.2	Analyse der Spezifikation	9
3.3	Entwicklung der Beispielanwendungen	10
3.4	Durchführung der Sicherheitstests	10
3.5	Verwendete Werkzeuge	11

4	Sicherheits- und Bedrohungsanalyse von WebAssembly	12
4.1	Das Bedrohungsmodell	12
4.1.1	Angreiferprofil und Angriffsziele	12
4.1.2	Definition der Angriffsoberflächen	13
4.2	Defensivmechanismen vs. Klassische Schwachstellen	13
4.2.1	Sandboxing und die Isolation des Host-Systems	13
4.2.2	Linearer Speicher und das Verhindern von Host-Exploitation	14
4.2.3	Validierung, Typsicherheit und Kontrollflussintegrität (CFI)	14
4.2.4	Die verbleibende Verwundbarkeit: Speicherfehler innerhalb des Moduls (Buffer Overflows, Use-after-Free, Integer Overflows)	15
4.3	Neue Angriffsvektoren und architektonische Grenzen	15
4.3.1	Mikroarchitektonische Angriffe (Side-Channel, Spectre)	15
4.3.2	Schwachstellen in der JavaScript-Wasm-Interaktionsschicht	16
4.3.3	Erweiterung der Angriffsoberfläche durch WASI-Schnittstellen . . .	16
5	Praktische Untersuchung von Sicherheitslücken	18
5.1	Testumgebung	18
5.1.1	Hardware und Software	18
5.1.2	Browser und Runtime-Umgebungen	18
5.2	Beispiel 1: Stack-Buffer-Overflow und Authentifizierungsumgehung	18
5.2.1	Implementierung	18
5.2.2	Durchführung	19
5.2.3	Ergebnisse	19
5.3	Beispiel 2: Use-After-Free und Heap-Speicherkontrolle	19
5.3.1	Implementierung	19
5.3.2	Durchführung	20
5.3.3	Ergebnisse	20
5.4	Beispiel 3: Integer-Overflow und Umgehung einer Längenprüfung	20
5.4.1	Implementierung	20
5.4.2	Durchführung	20
5.4.3	Ergebnisse	21
5.5	Beispiel 4: JS-WASM-Grenzvertrauen in Rust	21
5.5.1	Implementierung	21
5.5.2	Durchführung	21
5.5.3	Ergebnisse	22
5.6	Beispiel 5: AddressSanitizer als Gegenmaßnahme	22
5.6.1	Implementierung	22
5.6.2	Durchführung	22
5.6.3	Ergebnisse	22

6	Diskussion und Bewertung	23
6.1	Beantwortung der Forschungsfragen	23
6.2	Bewertung der Sicherheitsmechanismen	23
6.3	Grenzen der Untersuchung	23
6.4	Empfehlungen für Entwickler	23
7	Fazit und Ausblick	24
7.1	Zusammenfassung	24
7.2	Zukünftige Entwicklungen	24
7.3	Weiterführende Forschungsansätze	24

Abstract

1 Einleitung

1.1 Motivation

Die Webplattform hat sich in den vergangenen Jahren von einem Medium zur Darstellung statischer Inhalte zu einer vollwertigen Anwendungsplattform entwickelt. Mit dieser Entwicklung steigen die Anforderungen an Ausführungsleistung und Portabilität von Webanwendungen erheblich. WebAssembly (Wasm) wurde als Antwort auf diese Anforderungen entwickelt und seit 2019 als W3C-Standard etabliert [Wor19b]. Die Technologie ermöglicht die Ausführung von kompiliertem Code aus Sprachen wie C, C++ oder Rust im Browser, mit einer Performance, die sich der nativer Anwendungen annähert.

Der Einsatz von WebAssembly wächst rapide: Von interaktiven 3D-Anwendungen über Bildbearbeitungswerkzeuge bis hin zu serverseitigen Container-Laufzeitumgebungen findet die Technologie zunehmend Verbreitung in produktiven Systemen [PR25]. Diese Ausbreitung macht WebAssembly zu einem sicherheitsrelevanten Bestandteil moderner Software-Infrastrukturen.

WebAssembly wurde mit explizitem Fokus auf Sicherheit entworfen: Das Sandbox-Modell, das typischere Ausführungsmodell und die statische Validierung sind als inhärente Sicherheitsmechanismen konzipiert [Wor19a]. Gleichzeitig bringt die Technologie charakteristische Eigenschaften mit sich, die aus Sicherheitsperspektive kritisch zu betrachten sind. Da ein Großteil der eingesetzten Wasm-Module durch Kompilierung von C- oder C++-Code entsteht, werden klassische Speicherfehler dieser Sprachen in den WebAssembly-Kontext übertragen. Die Frage, welche Schutzwirkung die Wasm-Sandbox in diesen Fällen tatsächlich bietet und wo ihre Grenzen liegen, ist für Entwickler und Sicherheitsforscher gleichermaßen relevant.

1.2 Problemstellung

Trotz der sicherheitsorientierten Designziele von WebAssembly zeigt die Forschung, dass die Technologie nicht frei von Schwachstellen ist. Lehmann et al. [LKP20] demonstrieren, dass klassische Angriffstechniken der Binärsicherheit — darunter Buffer Overflows, Use-after-Free und Integer Overflows — im Wasm-Kontext weiterhin wirksam sind, auch wenn sich ihre Auswirkungen durch das Sandbox-Modell grundlegend verändern. Stiévenart et al. [SRG21] zeigen darüber hinaus, dass gängige Compiler-Toolchains wie Emscripten etablierte Schutzmechanismen wie Stack Canaries oder ASLR standardmäßig nicht aktivieren.

Auf der anderen Seite entstehen durch den Einsatz von WebAssembly neue Angriffsvektoren, die in rein JavaScript-basierten Anwendungen nicht in dieser Form auftreten: Die asymmetrische Vertrauensgrenze zwischen JavaScript-Host und Wasm-Modul eröffnet Angriffspfade, bei denen Speicherfehler im Modul über die Sandbox-Grenze hinaus zu JavaScript-Codeausführung führen können [Dra+25]. Zudem verstärkt WebAssembly durch seine Ausführungscharakteristik das Potenzial für mikroarchitektonische Side-Channel-Angriffe wie Spectre [PR25].

Es besteht damit eine Spannung zwischen dem Sicherheitsversprechen von WebAssembly und den in der Praxis beobachteten Schwachstellen. Eine fundierte Analyse, die beide

Seiten — Schutzwirkung und verbleibende Angriffsfläche — systematisch untersucht und durch praktische Demonstrationen belegt, stellt eine relevante Grundlage für den sicheren Einsatz der Technologie dar.

1.3 Zielsetzung

Ziel dieser Arbeit ist eine systematische Untersuchung der Sicherheitsaspekte von WebAssembly, die sowohl die eingebauten Defensivmechanismen als auch die verbleibenden und neu entstehenden Schwachstellenklassen analysiert. Die Untersuchung kombiniert eine theoretische Bedrohungsanalyse mit einer empirischen Komponente: Fünf selbst entwickelte Proof-of-Concept-Implementierungen demonstrieren konkrete Schwachstellen im Browser-Kontext und machen deren Auswirkungen unmittelbar nachvollziehbar.

Die Arbeit richtet sich an Entwickler, die WebAssembly in sicherheitskritischen Anwendungen einsetzen, sowie an Sicherheitsforschende, die das Bedrohungsmodell der Technologie einordnen möchten. Sie verfolgt keinen Anspruch auf Vollständigkeit aller bekannten Wasm-Schwachstellen, sondern auf eine fundierte, exemplarische Analyse ausgewählter, praxisrelevanter Schwachstellenklassen.

1.4 Forschungsfragen

Aus der Problemstellung und Zielsetzung ergeben sich die folgenden fünf Forschungsfragen, die im Verlauf der Arbeit beantwortet werden:

1. Welche Sicherheitsmechanismen implementiert WebAssembly standardmäßig?
2. Inwiefern schützt WebAssembly vor klassischen Webangriffen?
3. Welche neuen Angriffsvektoren entstehen durch den Einsatz von WebAssembly?
4. Welche praktischen Sicherheitslücken lassen sich demonstrieren?
5. Wie kann die Sicherheit von WebAssembly-Anwendungen verbessert werden?

1.5 Aufbau der Arbeit

Die Arbeit gliedert sich in sieben Kapitel. Kapitel 2 legt das theoretische und technische Fundament: Es beschreibt die Entstehung und Standardisierung von WebAssembly, dessen interne Architektur sowie die relevanten Sicherheitskonzepte der Webplattform. Kapitel 3 erläutert die methodische Vorgehensweise, insbesondere die Literaturrecherche und die Entwicklung der Proof-of-Concept-Implementierungen. In Kapitel 4 wird eine Bedrohungsanalyse durchgeführt: Es werden Angreiferprofile und Angriffsoberflächen definiert, die Defensivmechanismen von WebAssembly bewertet und neue Angriffsvektoren identifiziert. Kapitel 5 präsentiert die fünf praktischen Untersuchungen — Buffer Overflow, Use-after-Free, Integer Overflow, JS-Wasm-Grenzvertrauen sowie AddressSanitizer als Gegenmaßnahme — und dokumentiert deren Implementierung, Durchführung und Ergebnisse. Kapitel 6 diskutiert die Ergebnisse, beantwortet die Forschungsfragen explizit

und leitet Empfehlungen für Entwickler ab. Kapitel 7 fasst die Erkenntnisse der Arbeit zusammen und skizziert weiterführende Forschungsansätze.

2 Grundlagen

Das folgende Kapitel legt das theoretische und technische Fundament für die vorliegende Arbeit. Um die Sicherheitsaspekte von WebAssembly fundiert analysieren zu können, werden zunächst die historischen und konzeptionellen Ursprünge der Technologie sowie deren grundlegende Designziele erörtert.

Darüber hinaus wird die interne Systemarchitektur von WebAssembly detailliert betrachtet, wobei das Speichermodell, das Ausführungsprinzip und die Host-Integration im Fokus stehen. Da WebAssembly-Anwendungen in der Praxis meist innerhalb bestehender Web-Infrastrukturen operieren, schließt das Kapitel mit einer Betrachtung der etablierten Sicherheitskonzepte der Webplattform ab, die den Ausführungsrahmen für die Technologie vorgeben.

2.1 WebAssembly

Als moderne Bytecode-Technologie hat WebAssembly die Ausführung von Code im Web grundlegend revolutioniert und stellt mittlerweile eine vollwertige Alternative zu rein JavaScript-basierten Architekturen dar. Für ein umfassendes Verständnis ist es notwendig, die Technologie sowohl in ihrem historischen Kontext als auch in Bezug auf ihre strategische Ausrichtung zu betrachten. Im Folgenden werden daher die Entstehungsgeschichte und Standardisierung von WebAssembly nachgezeichnet, die primären Designziele analysiert und die Entwicklung des technologischen Ökosystems von einer reinen Browser-Erweiterung hin zu einer universellen Runtime-Umgebung aufgezeigt.

2.1.1 Einordnung, Entstehung und Standardisierung

WebAssembly (Wasm) wurde 2015 als binäres Ausführungsformat für Webanwendungen vorgestellt [Eic15]. Die Entwicklung erfolgte als Reaktion auf die Grenzen von asm.js [Moz13]. Ziel der Entwicklung war es, eine kompakte und effizient dekodierbare Repräsentation bereitzustellen, die eine leistungsfähige Ausführung in Browsern ermöglicht [Web24].

Im Jahr 2017 gründete das W3C die WebAssembly Working Group, um die Standardisierung von WebAssembly voranzutreiben [W3C17]. Ziel dieser Initiative war die gemeinsame Ausarbeitung einer Spezifikation für die Technologie. Im Dezember 2019 wurde die WebAssembly Core Specification als W3C Recommendation veröffentlicht und damit zum offiziellen Standard des World Wide Web Consortiums (W3C) [Wor19b].

2.1.2 Designziele von WebAssembly

Die Entwicklung von WebAssembly orientierte sich an drei zentralen Designzielen. Erstens soll eine hohe Ausführungsleistung erreicht werden, die sich möglichst an der nativen Performance von Anwendungen orientiert. Zweitens steht die Sicherheit im Vordergrund: WebAssembly-Code wird vor der Ausführung validiert und innerhalb einer isolierten Sandbox-Umgebung ausgeführt, um Speicherfehler und unerwünschte Systemzugriffe

zu verhindern. Drittens verfolgt die Technologie das Ziel der Portabilität, sodass derselbe Code unabhängig von Betriebssystem, Hardwarearchitektur oder Programmiersprache eingesetzt werden kann [Wor19a].

2.1.3 Ökosystem und Einsatzbereiche

Ursprünglich wurde WebAssembly primär für die effiziente Ausführung von aus C- und C++-Code kompilierten Anwendungen entwickelt, insbesondere im Kontext von Toolchains wie Emscripten [Ems].

Im weiteren Verlauf entstand ein wachsendes Ökosystem, das die Nutzung weiterer Programmiersprachen wie Rust und Python durch entsprechende Tooling- und Runtime-Unterstützung ermöglicht, beispielsweise durch das Rust-WebAssembly-Tooling und Pyodide [Rus; Pyo].

Dadurch entwickelte sich WebAssembly über den ursprünglichen Einsatz im Browser hinaus zu einer plattformübergreifenden Runtime-Umgebung, die auch außerhalb von Webanwendungen eingesetzt wird, etwa durch WASI-konforme Runtime-Implementierungen [Wor19a; Byt].

2.2 Architektur von WebAssembly

Die Architektur von WebAssembly basiert auf einem modularen, sicheren und plattformunabhängigen Ausführungsmodell, das in der W3C Core Specification definiert ist [Wor19a]. Ziel des Designs ist es, eine effiziente, portable und deterministische Ausführung von kompiliertem Code in unterschiedlichen Runtime-Umgebungen zu ermöglichen.

2.2.1 Modulkonzept

Die grundlegende Einheit in WebAssembly ist das Modul. Ein Modul stellt eine abgeschlossene, binäre Repräsentation eines Programms dar und enthält sowohl ausführbaren Code als auch strukturierte Metadaten. Dazu gehören unter anderem Funktionsdefinitionen, Import- und Export-Schnittstellen sowie Definitionen für Speicherbereiche und Tabellen. Module sind unabhängig voneinander ausführbar und können in verschiedene Host-Umgebungen geladen und instanziiert werden [Wor19a].

2.2.2 Ausführungsmodell

WebAssembly basiert auf einem Stack-basierten Ausführungsmodell. Instruktionen arbeiten nicht direkt auf Registern, sondern verwenden einen Operand-Stack, auf den Werte gelegt und von dem sie wieder entnommen werden. Dieses Modell trägt zur Vereinfachung der Semantik und zur Portabilität der Ausführung bei, da keine architekturspezifischen Registermodelle vorausgesetzt werden [Wor19a].

2.2.3 Speichermodell und Isolation

Das Speichermodell von WebAssembly basiert auf einem linearen, zusammenhängenden Speicherbereich. Dieser Speicher wird als Byte-Array abstrahiert und verhindert direkten Zugriff auf nativen Prozessspeicher oder Hardwareadressen. Dadurch wird eine starke Isolation zwischen WebAssembly-Code und der Host-Umgebung gewährleistet. Speicherzugriffe erfolgen ausschließlich innerhalb definierter Grenzen und werden zur Laufzeit überprüft [Wor19a].

2.2.4 Typisierung und Validierung

Vor der Ausführung wird jedes WebAssembly-Modul statisch validiert. Dabei wird unter anderem geprüft, ob Typen konsistent verwendet werden und ob keine ungültigen Instruktionsfolgen vorliegen. Dieses Validierungsverfahren stellt sicher, dass nur wohlgeformte Module ausgeführt werden, wodurch Runtime-Fehler und unsichere Speicherzugriffe bereits vor der Ausführung ausgeschlossen werden können [Wor19a].

2.2.5 Sandbox-Modell

WebAssembly-Code wird innerhalb einer isolierten Ausführungsumgebung (Sandbox) ausgeführt und besitzt keinen direkten Zugriff auf Betriebssystemressourcen oder den Speicher des Hosts. Jegliche Interaktion mit der Außenwelt erfolgt ausschließlich über explizit definierte Schnittstellen, die vom Host bereitgestellt werden [Wor19a].

2.2.6 Host-Integration und Laufzeitumgebungen

Die Integration in Host-Umgebungen erfolgt über definierte Schnittstellen, beispielsweise die JavaScript-API in Browsern oder WASI (WebAssembly System Interface) in serverseitigen Szenarien. Während die JavaScript-API primär die Interoperabilität im Webkontext ermöglicht, erweitert WASI WebAssembly um standardisierte Systemaufrufe für Dateisysteme, Netzwerkzugriffe und Prozessverwaltung. Dadurch kann WebAssembly auch außerhalb des Browsers als portable Runtime-Umgebung eingesetzt werden [Wor19a; Byt].

2.3 Sicherheitskonzepte im Web

Web-Anwendungen werden durch mehrere komplementäre Sicherheitsmechanismen der Webplattform geschützt. Diese Mechanismen definieren, wie Ressourcen zwischen unterschiedlichen Webseiten isoliert und kontrolliert ausgetauscht werden können. Im Kontext von WebAssembly sind diese Konzepte besonders relevant, da WebAssembly-Module typischerweise innerhalb des Web-Sicherheitsmodells des Browsers ausgeführt werden.

2.3.1 Same-Origin Policy

Die Same-Origin Policy (SOP) stellt einen grundlegenden Sicherheitsmechanismus des Webs dar. Sie verhindert, dass Dokumente oder Skripte einer Origin ohne explizite Berechtigung auf Ressourcen einer anderen Origin zugreifen können. Eine Origin wird dabei

durch das Protokoll (Schema), den Hostnamen und den Port definiert. Durch diese Isolation wird verhindert, dass sensible Daten zwischen unterschiedlichen Webseiten unkontrolliert ausgelesen oder manipuliert werden können [Bar11].

Im Kontext von WebAssembly gilt die Same-Origin Policy ebenfalls für den Zugriff auf Web-APIs und Ressourcen, da WebAssembly-Module innerhalb derselben Origin und desselben Ausführungskontexts im Browser ausgeführt werden.

2.3.2 Content Security Policy

Die Content Security Policy (CSP) ergänzt die Same-Origin Policy durch ein deklaratives Sicherheitsmodell, das die zulässigen Quellen für Ressourcen wie Skripte, Styles oder externe Inhalte einschränkt [W3C23]. Durch serverseitig definierte Richtlinien kann festgelegt werden, welche Inhalte im Browser geladen und ausgeführt werden dürfen.

CSP dient insbesondere dem Schutz vor Angriffen wie Cross-Site-Scripting (XSS), indem die Ausführung nicht autorisierter Skripte verhindert wird. Da WebAssembly-Module in Webanwendungen typischerweise über JavaScript initialisiert und instanziiert werden, unterliegen sie ebenfalls den durch CSP definierten Restriktionen.

2.3.3 JavaScript-Sicherheitsmodell

Das JavaScript-Sicherheitsmodell bildet die Grundlage für die Ausführung von WebAssembly im Browser, da WebAssembly in der Regel über JavaScript-APIs geladen und kontrolliert wird [ECM25]. Dieses Modell basiert auf dem Prinzip der isolierten Ausführung innerhalb eines Web-Realm-Kontexts, in dem Skripte einer Origin zugeordnet sind.

WebAssembly selbst erweitert dieses Modell nicht, sondern integriert sich in die bestehenden Sicherheitsmechanismen des Browsers. Dadurch bleibt die Kontrolle über Ressourcen und Systemzugriffe vollständig im Host (Browser), während WebAssembly innerhalb derselben Sandbox- und Berechtigungskontexte wie JavaScript ausgeführt wird.

3 Methodik

Das vorliegende Kapitel beschreibt die methodische Vorgehensweise der Arbeit. Ziel ist es, die gewählten Untersuchungsansätze zu begründen und die Reproduzierbarkeit der erzielten Ergebnisse sicherzustellen. Die Untersuchung kombiniert eine selektive Literaturrecherche mit einer empirischen Komponentenanalyse in Form selbst entwickelter Proof-of-Concept-Implementierungen.

3.1 Literaturrecherche

Die Literaturrecherche erfolgte als selektive Recherche mit dem Ziel, einen fundierten und aktuellen Überblick über den Stand der Forschung zu Sicherheitsaspekten von WebAssembly zu gewinnen. Als primäre Quellen wurden wissenschaftliche Datenbanken herangezogen, darunter ACM Digital Library, IEEE Xplore, USENIX und arXiv. Ergänzend wurden offizielle Spezifikationsdokumente des W3C sowie Dokumentationen relevanter Toolchain-Projekte einbezogen.

Die Suche konzentrierte sich auf Veröffentlichungen ab dem Jahr 2019, da die Standardisierung der WebAssembly Core Specification als W3C Recommendation in diesem Jahr einen zentralen Entwicklungsschritt markiert. Für besonders aktuelle Themengebiete wurden auch Preprints berücksichtigt, sofern eine ausreichende inhaltliche Qualität erkennbar war [Cor+26].

Als primäre Suchbegriffe wurden unter anderem *WebAssembly security*, *Wasm memory safety*, *Wasm attack surface*, *binary security WebAssembly* sowie *WebAssembly sandbox* verwendet. Bei der Auswahl relevanter Quellen wurde Peer-reviewed-Literatur bevorzugt, insbesondere Beiträge von renommierten Sicherheitskonferenzen wie dem USENIX Security Symposium und Workshops der ACM. Review-Artikel und Surveys dienten dabei zur Einordnung des Forschungsfeldes und zur Identifikation weiterer Primärquellen [PR25; Jun+24].

3.2 Analyse der Spezifikation

Neben der Sekundärliteratur wurde die WebAssembly Core Specification des W3C als primäre normative Quelle herangezogen [Wor19a]. Dieser Ansatz ist methodisch begründet, da ein erheblicher Teil der Fachliteratur die Spezifikation lediglich paraphrasiert, ohne die zugrundeliegenden normativen Anforderungen direkt zu referenzieren. Für Aussagen über das definierte Verhalten des Speichermodells, des Validierungsverfahrens und des Sandbox-Designs wurde daher stets die Spezifikation selbst konsultiert.

Die Analyse konzentrierte sich auf die Abschnitte zu Memory, Execution, Validation sowie auf die Beschreibung des Modulformats. Ziel war es, Aussagen über Sicherheitseigenschaften von WebAssembly auf der normativen Grundlage zu fundieren, anstatt sie ausschließlich aus interpretativer Sekundärliteratur abzuleiten.

3.3 Entwicklung der Beispielanwendungen

Zur empirischen Untersuchung konkreter Schwachstellenklassen wurden fünf eigenständige Proof-of-Concept-Implementierungen (PoCs) entwickelt. Die Entscheidung für eine praktisch-demonstrative Methodik ist dadurch begründet, dass die untersuchten Schwachstellenklassen erst durch tatsächliche Ausführbarkeit im Browser vollständig nachvollziehbar werden: Eine rein theoretische Beschreibung kann nicht zeigen, ob und wie ein Speicherfehler innerhalb eines Wasm-Moduls tatsächlich ausnutzbar ist und wie sich die Sandbox-Isolation in der Praxis verhält.

Die Auswahl der untersuchten Schwachstellenklassen orientiert sich an Corrias et al. [Cor+26], die das systematische Auftreten klassischer binärer Speicherfehler in Wasm-basierten Webanwendungen analysieren. Konkret wurden Buffer Overflows, Use-after-Free-Fehler und Integer Overflows untersucht. Ergänzend wurde ein vierter PoC entwickelt, der eine eigenständige Schwachstellenkategorie an der Grenze zwischen JavaScript-Host und Wasm-Modul adressiert und nicht auf einem Speicherfehler im engeren Sinne beruht. Ein fünfter PoC demonstriert den Einsatz des AddressSanitizers als Gegenmaßnahme und dient dem direkten Vergleich mit dem uninstrumentierten Äquivalent.

Die Wahl von C als primärer Quellsprache ist durch dessen dominante Stellung im realen Wasm-Ökosystem motiviert: Ein erheblicher Anteil produktiv eingesetzter Wasm-Module entsteht durch Kompilierung von C- oder C++-Code mittels Emscripten [Ems]. Rust wurde für den vierten PoC gewählt, um zu zeigen, dass sprachseitige Speichersicherheit keine hinreichende Schutzmaßnahme an der Host-Grenze darstellt.

Bewusst ausgeklammert wurden automatisierte Fuzzing-Kampagnen gegen Real-World-Anwendungen, wie sie etwa von Draissi et al. [Dra+25] durchgeführt wurden, sowie Angriffe gegen native Wasm-Runtime-Implementierungen außerhalb des Browsers. Der Fokus dieser Arbeit liegt auf dem Verhalten innerhalb der Browser-Ausführungsumgebung.

3.4 Durchführung der Sicherheitstests

Die Sicherheitstests wurden als manuelle, gezielte Demonstration durchgeführt. Für jeden PoC wurde ein spezifischer Angriffspfad vordefiniert und im Browser nachvollzogen. Als Beobachtungsinstrumente dienten die Browser Developer Tools (Konsole, Speicherinspektor, Netzwerk-Tab), ergänzt durch direkten JavaScript-Zugriff auf `WebAssembly.Memory`, um Speicherzustände vor und nach dem Angriff zu inspizieren.

Das Ziel der Tests war nicht die Messung von Angriffserfolgswahrscheinlichkeiten über eine Stichprobe von Anwendungen, sondern die qualitative Demonstration des Verhaltens der Wasm-Runtime bei konkreten Schwachstellenklassen. Insbesondere wurde untersucht, ob und wie die Sandbox-Isolation eine Eskalation auf die Host-Umgebung verhindert, während modulinterne Schwachstellen dennoch ausnutzbar bleiben. Für Beispiel 5 wurde zusätzlich das ASan-Laufzeitverhalten über den `printErr`-Kanal des Emscripten-Moduls im Browser beobachtet.

3.5 Verwendete Werkzeuge

- **Emscripten (emsdk)** – Toolchain zur Kompilierung von C-Quellcode zu WebAssembly; basiert intern auf LLVM/Clang.
- **Rust-Toolchain** mit Compilierziel `wasm32-unknown-unknown` sowie `wasm-bindgen` – für das Rust-basierte PoC-Modul.
- **Docker** – Containerisierung aller PoCs für eine reproduzierbare Ausführungsumgebung.
- **Python http.server** – lokale Bereitstellung der Webanwendungen innerhalb der Docker-Container.
- **Google Chrome / Mozilla Firefox** – primäre Ausführungs- und Beobachtungsumgebung; Browser Developer Tools für Konsolenausgabe, Speicherinspektion und Laufzeitbeobachtung.
- **LLVM AddressSanitizer (ASan) via Emscripten** – Laufzeitinstrumentierung zur Erkennung von Heap-Speicherfehlern.

4 Sicherheits- und Bedrohungsanalyse von WebAssembly

Das folgende Kapitel analysiert die Sicherheitseigenschaften von WebAssembly aus einer bedrohungsanalytischen Perspektive. Aufbauend auf den in Kapitel 2 beschriebenen Architekturmerkmalen wird zunächst ein Bedrohungsmodell entwickelt, das relevante Angreiferprofile und Angriffsflächen definiert. Anschließend werden die eingebauten Defensivmechanismen von WebAssembly gegenüber klassischen Schwachstellenklassen bewertet und deren Grenzen identifiziert. Das Kapitel schließt mit einer Betrachtung neuartiger Angriffsvektoren, die spezifisch durch den Einsatz von WebAssembly entstehen oder verstärkt werden. Die in diesem Kapitel identifizierten Schwachstellenklassen bilden die theoretische Grundlage für die empirischen Untersuchungen in Kapitel 5.

4.1 Das Bedrohungsmodell

4.1.1 Angreiferprofil und Angriffsziele

Für eine fundierte Sicherheitsanalyse ist zunächst die Frage zu beantworten, welche Angreiferklassen im Kontext von WebAssembly-Anwendungen relevant sind. Basierend auf der Literatur lassen sich zwei primäre Angreiferprofile unterscheiden [PR25; Jun+24].

Das erste Profil umfasst den *externen Netzwerkangreifer*, der keine privilegierten Zugriffsrechte auf die Zielanwendung besitzt, jedoch Eingaben kontrollieren kann, die von der Webanwendung verarbeitet werden. Typische Angriffsvektoren sind manipulierte Formularfelder, präparierte API-Anfragen oder Nutzdaten, die über Netzwerkschnittstellen an das Wasm-Modul weitergeleitet werden. Das primäre Angriffsziel ist die Manipulation der Modullogik, etwa durch Ausnutzung von Speicherfehlern, um Authentifizierungslogik zu umgehen oder geschützte Daten zu extrahieren. Draissi et al. [Dra+25] zeigen in einer großangelegten empirischen Studie, dass 77,81 % der untersuchten Domains Daten aus potenziell Angreifer-kontrollierten Quellen vollständig vertrauen, was dieses Profil als besonders praxisrelevant ausweist.

Das zweite Profil beschreibt den *bösartigen Wasm-Code-Angreifer*, bei dem ein manipuliertes oder kompromittiertes Wasm-Modul selbst als Angriffswerkzeug eingesetzt wird, etwa durch Supply-Chain-Angriffe über kompromittierte Abhängigkeiten oder durch das Einschleusen von Wasm-Code über eine bereits bestehende Schwachstelle. Das Angriffsziel ist in diesem Fall der Ausbruch aus der Browser-Sandbox oder die Kompromittierung anderer Ressourcen innerhalb des Browser-Kontexts.

Unabhängig vom konkreten Angreiferprofil lassen sich vier generische Angriffsziele identifizieren: (1) Umgehung von Sicherheits- und Zugriffslogik innerhalb des Moduls, (2) Exfiltration vertraulicher Daten aus dem linearen Speicher, (3) Erlangung von JavaScript-Codeausführung durch Ausnutzung impliziten Vertrauens an der JS-Wasm-Grenze sowie (4) Missbrauch von WebAssembly als Vehikel für Cryptomining oder andere unerwünschte rechenintensive Aktivitäten [PR25].

4.1.2 Definition der Angriffsoberflächen

Die Angriffsoberfläche von WebAssembly-Anwendungen lässt sich in drei konzeptionelle Schichten gliedern, die in der Literatur als grundlegendes Strukturierungsrahmenwerk verwendet werden [Jun+24].

Die erste Schicht bildet die *modulinterne Angriffsoberfläche*. Sie umfasst alle Schwachstellen, die innerhalb des linearen Speichers des Wasm-Moduls entstehen und ausnutzbar sind, ohne die Sandbox-Grenze zu überschreiten. Hierzu zählen klassische binäre Speicherfehler wie Buffer Overflows, Use-after-Free und Integer Overflows, die aus unsicheren Programmierpraktiken in den Quellsprachen resultieren und durch die Wasm-Kompilierung nicht eliminiert werden.

Die zweite Schicht ist die *JS-Wasm-Grenze* (Host-Boundary). Da JavaScript-Code vollständigen Lese- und Schreibzugriff auf `WebAssembly.Memory` besitzt, ist diese Schnittstelle asymmetrisch vertrauenswürdig: Das Wasm-Modul kann alle vom Host übergebenen Werte — Zeiger, Längenangaben, Flags — nicht eigenständig als vertrauenswürdig verifizieren. Gleichzeitig vertrauen viele JS-Anwendungen den Ausgaben des Wasm-Moduls implizit und leiten diese in sicherheitssensitive Funktionen wie `eval()` oder `innerHTML` weiter, was Angriffe über Speicherfehler im Modul auf den JS-Kontext ermöglicht [Dra+25].

Die dritte Schicht umfasst die *Browser-Umgebung und Plattform*. Hier operiert Wasm innerhalb desselben Sicherheitskontexts wie JavaScript und unterliegt denselben Einschränkungen durch SOP, CSP und das JS-Sicherheitsmodell. Gleichzeitig kann Wasm durch hochpräzise Timing-Mechanismen mikroarchitektonische Seitenkanalangriffe ermöglichen, die über die Modulgrenzen hinaus wirken.

4.2 Defensivmechanismen vs. Klassische Schwachstellen

4.2.1 Sandboxing und die Isolation des Host-Systems

Der zentrale Sicherheitsgewinn von WebAssembly gegenüber nativen Anwendungen liegt in der strikten Isolation des Moduls von der Host-Umgebung. WebAssembly-Code besitzt keinen direkten Zugriff auf den nativen Prozessspeicher des Browsers, auf Betriebssystem-Syscalls oder auf Hardwareadressen. Jegliche Interaktion mit der Außenwelt erfolgt ausschließlich über explizit importierte Funktionen, die der Host dem Modul bereitstellt [Wor19a].

Lehmann et al. [LKP20] analysieren in ihrer grundlegenden Studie *“Everything Old is New Again”*, wie sich klassische Exploit-Techniken aus dem Bereich der Binärsicherheit auf den WebAssembly-Kontext übertragen lassen. Ihre Kernaussage ist, dass das Sandbox-Modell die Wirkung von Exploits grundlegend verändert: Während ein Buffer Overflow in nativem Code über die Kontrolle der Rücksprungadresse auf dem Stack zu beliebiger Codeausführung führen kann (Return-Oriented Programming, ROP), ist dieser Angriffspfad in WebAssembly nicht möglich, da der Call-Stack nicht Teil des adressierbaren linearen Speichers ist und Rücksprungadressen für den Modulcode nicht direkt manipulierbar sind.

Die Sandbox schützt damit primär den Host: Eine Schwachstelle im Wasm-Modul führt nicht zwangsläufig zur Kompromittierung des Browsers oder des Betriebssystems. Die-

ser Schutzgewinn ist ein wesentlicher Designvorteil gegenüber der direkten Ausführung nativen Codes im Browser-Kontext.

4.2.2 Linearer Speicher und das Verhindern von Host-Exploitation

Das lineare Speichermodell von WebAssembly, wie in Abschnitt 2.2.3 beschrieben, stellt den gesamten adressierbaren Speicher des Moduls als zusammenhängenden Byte-Array im JavaScript-Heap dar. Diese Abstraktion hat weitreichende Sicherheitskonsequenzen. Überläufe jeglicher Art — Stack-basierte oder Heap-basierte Buffer Overflows — bleiben innerhalb dieses Arrays und können damit nicht auf den nativen Prozessspeicher des Hosts zugreifen. Die Grenze des linearen Speichers wird zur Laufzeit durch Bounds-Checking erzwungen; ein Zugriff außerhalb des definierten Bereichs führt zu einem Trap, nicht zu undefiniertem Verhalten mit systemeigenen Konsequenzen [Wor19a].

Gleichzeitig bedeutet dieses Modell, dass der lineare Speicher selbst keine internen Schutzmechanismen besitzt. Stack-Daten, Heap-Daten und globale Variablen liegen im selben flachen Adressraum. Compiler-seitige Schutzmaßnahmen, die in nativen Anwendungen standardmäßig eingesetzt werden — darunter Stack Canaries, Address Space Layout Randomization (ASLR) und Non-Executable-Speicherbereiche (NX/DEP) — sind in Wasm-Binaries typischerweise nicht vorhanden [SRG21]. Die Abwesenheit dieser Schutzmechanismen ermöglicht es, dass Angreifer innerhalb des linearen Speichers zuverlässig vorher sagen können, wie Datenstrukturen angeordnet sind, und gezielte Überläufe durchführen können.

4.2.3 Validierung, Typsicherheit und Kontrollflussintegrität (CFI)

Bevor ein Wasm-Modul instanziiert wird, durchläuft es einen statischen Validierungsprozess, der in der W3C Core Specification formal definiert ist [Wor19a]. Dieser Prozess prüft die Wohlgeformtheit des Moduls: Typen müssen konsistent verwendet werden, Instruktionsfolgen müssen semantisch gültig sein, und alle referenzierten Funktionen, Tabellen und Speicherbereiche müssen korrekt deklariert sein. Module, die diesen Validierungsprozess nicht bestehen, werden abgelehnt und nicht ausgeführt.

Ein wichtiger Aspekt der Kontrollflussintegrität in WebAssembly ist die Beschränkung indirekter Funktionsaufrufe. Der `call_indirect`-Instruktion können nur Einträge aus der Funktionstabelle des Moduls als Ziel dienen. Dies verhindert, dass ein Angreifer durch Manipulation von Funktionszeigern im linearen Speicher beliebige Adressen als Sprungziel verwenden kann — ein in nativem Code häufig genutzter Exploit-Pfad. Diese Eigenschaft stellt eine inhärente Form von CFI dar [LKP20].

Die Grenzen dieser Schutzmaßnahmen liegen allerdings auf der Datenflussebene: Die Validierung prüft den Kontrollfluss und die Typen, nicht aber die semantische Korrektheit der verarbeiteten Werte. Arithmetische Überläufe, fehlerhafte Längenberechnungen oder die Dereferenzierung ungültiger Zeiger innerhalb des linearen Speichers sind keine Verletzungen des Wasm-Typsystems und werden daher nicht durch die statische Validierung erkannt.

4.2.4 Die verbleibende Verwundbarkeit: Speicherfehler innerhalb des Moduls (Buffer Overflows, Use-after-Free, Integer Overflows)

Trotz der beschriebenen Defensivmechanismen gilt die zentrale Erkenntnis von Lehmann et al. [LKP20]: WebAssembly verlagert klassische Speicherfehler in einen neuen Kontext, verhindert sie jedoch nicht. Programme, die in unsicheren Sprachen wie C geschrieben wurden und klassische Speicherfehler enthalten, bringen diese Fehler beim Kompilieren nach WebAssembly mit. Die Wasm-Runtime erkennt und verhindert diese Fehler nicht — sie sind innerhalb des Sandboxing-Modells unsichtbar, solange sie keine Sandbox-Grenzen überschreiten.

Konkret bedeutet dies: Ein Buffer Overflow überschreibt benachbarte Daten im linearen Speicher; ein Use-after-Free greift auf freigegebene Heap-Blöcke zu, die der Allocator möglicherweise bereits neu vergeben hat; ein Integer Overflow führt zu fehlerhaften Längen- oder Größenberechnungen. In allen drei Fällen liegt das Fehlverhalten vollständig innerhalb des Wasm-Moduls und ist für die umgebende Sandbox nicht erkennbar. Die Wirkung kann dennoch schwerwiegend sein, da modulinterne Sicherheitslogik — Authentifizierungsprüfungen, Zugriffskontrollen, kryptographische Operationen — durch diese Fehler direkt kompromittiert werden kann.

Stiévenart et al. [SRG21] untersuchen diesen Aspekt aus der Perspektive fehlender Compiler-Schutzmaßnahmen und stellen fest, dass gängige Toolchains wie Emscripten diese Schutzmechanismen standardmäßig nicht aktivieren. Die Verantwortung für die Absicherung liegt damit vollständig beim Entwickler des Quellcodes und der Konfiguration der Toolchain.

4.3 Neue Angriffsvektoren und architektonische Grenzen

4.3.1 Mikroarchitektonische Angriffe (Side-Channel, Spectre)

WebAssembly ermöglicht die präzise Messung von Zeitintervallen, die für mikroarchitektonische Side-Channel-Angriffe wie Spectre notwendig sind. Der Spectre-Angriff nutzt spekulative Ausführung moderner Prozessoren, um Speicherinhalte außerhalb des eigenen Adressraums durch Messung von Cache-Timing-Unterschieden zu inferieren. Für diesen Angriff ist ein hochpräziser Timer erforderlich [PR25].

WebAssembly bietet in Verbindung mit der JavaScript-API `performance.now()` und insbesondere mit `SharedArrayBuffer` einen solchen hochpräzisen Timer: `SharedArrayBuffer` ermöglicht die Implementierung eines Shared-Memory-Counters, der als autonomer Taktgeber fungieren kann und damit eine Timer-Auflösung im Nanosekunden-Bereich erreicht. Als Reaktion auf die Veröffentlichung von Spectre im Jahr 2018 wurde `SharedArrayBuffer` temporär in allen gängigen Browsern deaktiviert. Heute ist die API nur noch in Kontexten verfügbar, die explizit als Cross-Origin-isoliert deklariert sind, was die HTTP-Response-Header `Cross-Origin-Opener-Policy` (COOP) und `Cross-Origin-Embedder-Policy` (COEP) erfordert.

Für WebAssembly bedeutet dies, dass das Angriffspotenzial durch diese Plattformmaßnahmen erheblich eingeschränkt, aber nicht vollständig beseitigt wurde. Perrone und Romano [PR25] ordnen mikroarchitektonische Angriffe als eine eigenständige Sicherheitska-

tegorie im Wasm-Ökosystem ein und stellen fest, dass WebAssembly durch seine Ausführungscharakteristik — hohe Ausführungsgeschwindigkeit, deterministisches Speichermodell, Kompilierung zu nativem Code — die Effektivität solcher Angriffe gegenüber reinem JavaScript erhöht.

4.3.2 Schwachstellen in der JavaScript-Wasm-Interaktionsschicht

Die Grenze zwischen JavaScript-Host und Wasm-Modul ist eine strukturell asymmetrische Vertrauensgrenze. Auf der einen Seite besitzt JavaScript durch die `WebAssembly.Memory`-API vollständigen Lese- und Schreibzugriff auf den linearen Speicher des Moduls. Das Modul selbst hat hingegen keinen Mechanismus, um die Korrektheit oder Vertrauenswürdigkeit der vom Host übergebenen Parameter — Zeiger, Längenangaben, Flags — eigenständig zu verifizieren. Alle vom Host übergebenen Werte müssen daher aus Modulsicht als potenziell feindlich betrachtet werden.

Diese strukturelle Eigenschaft führt zu zwei Klassen von Schwachstellen. Erstens kann ein Angreifer, der Eingaben in die Webanwendung kontrolliert, diese über JavaScript an das Wasm-Modul weitergeben und dabei absichtlich fehlerhafte Parameter übergeben — etwa überhöhte Längenangaben oder manipulierte Zeiger auf sicherheitskritische Bereiche des linearen Speichers. Das Modul führt Operationen mit diesen Parametern durch, ohne ihre Gültigkeit zu prüfen, sofern keine explizite Validierung im Modulcode vorgesehen ist.

Zweitens identifizieren Draissi et al. [Dra+25] eine komplementäre Angriffsvariante: Zahlreiche JavaScript-Anwendungen vertrauen den Ausgaben des Wasm-Moduls implizit und übergeben diese direkt an sicherheitssensitive Funktionen wie `eval()` oder `innerHTML`. Ein Angreifer, der durch einen Speicherfehler im Modul den Inhalt des linearen Speichers kontrolliert, kann so über die JS-Wasm-Grenze hinaus JavaScript-Code zur Ausführung bringen und damit ein Cross-Site-Scripting (XSS) erreichen. Diese Angriffskette verbindet modulinterne Speicherfehler mit schwerwiegenden Konsequenzen auf Anwendungsebene und zeigt, dass die Sandbox-Isolation des Moduls kein ausreichender Schutz ist, wenn das umgebende System den Modulausgaben blind vertraut.

4.3.3 Erweiterung der Angriffsoberfläche durch WASI-Schnittstellen

Das WebAssembly System Interface (WASI) erweitert WebAssembly um standardisierte Systemschnittstellen für den Einsatz außerhalb des Browsers, darunter Dateisystemzugriffe, Netzwerkkommunikation und Prozesskontrolle. Während das Browser-Sandbox-Modell den Systemzugriff auf das von der JavaScript-API Erlaubte beschränkt, öffnet WASI einen deutlich breiteren Zugriffspfad auf Betriebssystemressourcen. Dies verändert das Bedrohungsmodell für Wasm-basierte Server- und Container-Anwendungen grundlegend [Byt].

Yu et al. [Yu+25] analysieren im Rahmen des USENIX Security Symposiums 2025 die Ressourcenisolierungs-Angriffsfläche von WebAssembly-Containern und zeigen, dass WASI-basierte Runtime-Umgebungen Schwachstellen bei der Ressourcenisolierung aufweisen, die eine unbeabsichtigte Interaktion zwischen Wasm-Containern ermöglichen. Die Studie deckt auf, dass bestehende Implementierungen anfällig für Angriffe sind, bei denen ein Wasm-Modul über WASI-Schnittstellen Ressourcen eines anderen Moduls beeinflusst oder ausliest, obwohl eine strikte Isolation vorgesehen ist.

Die Implikation für die Sicherheitsbewertung von WebAssembly ist damit zweischichtig: Im Browser-Kontext sind die Schnittstellen auf die JavaScript-API beschränkt, was die Angriffsfläche eng begrenzt. Im WASI-Kontext nähert sich die Angriffsfläche der eines klassischen nativen Prozesses an, mit dem Unterschied, dass die Sandbox noch eine zusätzliche Isolationsschicht bietet — sofern deren Implementierung fehlerfrei ist. Die PoC-Implementierungen der vorliegenden Arbeit beschränken sich bewusst auf den Browser-Kontext; die WASI-Thematik wird als weiterführender Forschungsansatz in Kapitel 7 aufgegriffen.

5 Praktische Untersuchung von Sicherheitslücken

Die folgenden Abschnitte beschreiben fünf selbst entwickelte Proof-of-Concept-Implementierungen (PoCs), die konkrete Schwachstellenklassen im Kontext von WebAssembly demonstrieren. Die Auswahl der untersuchten Schwachstellenklassen orientiert sich dabei an der Arbeit von Corrias et al. [Cor+26], die einen systematischen Überblick über das Auftreten klassischer binärer Speicherfehler in WASM-basierten Webanwendungen liefert. Jeder PoC besteht aus einem in C oder Rust geschriebenen Modul, das mit Emscripten bzw. dem Rust-WebAssembly-Toolchain zu WASM kompiliert wird, sowie einer browserseitigen Runtime-Umgebung aus HTML und JavaScript. Die Ausführung erfolgt vollständig im Browser ohne serverseitige Logik, sodass die Interaktion zwischen Host-Umgebung und WASM-Modul direkt beobachtet werden kann. Sämtliche PoCs sind reproduzierbar über Docker-Container ausführbar.

5.1 Testumgebung

5.1.1 Hardware und Software

Alle PoCs wurden auf einem handelsüblichen x86-64-System unter Arch Linux entwickelt und getestet. Als Compiler-Toolchain kam Emscripten (emsdk, Basis-Image `emscripten/emsdk:latest`) zum Einsatz, das intern LLVM/Clang verwendet und C-Quellcode zu WASM kompiliert. Das Rust-Beispiel wurde mit der stable Rust-Toolchain und dem Compilierziel `wasm32-unknown-unknown` gebaut. Die Bereitstellung der Webanwendungen erfolgte über den in Python integrierten HTTP-Server (`python3 -m http.server 8080`), der als CMD-Anweisung im jeweiligen Docker-Image hinterlegt ist.

5.1.2 Browser und Runtime-Umgebungen

Als primäre Ausführungsumgebung diente Google Chrome in der aktuellen stabilen Version. Alle beschriebenen Beobachtungen sind in gleichem Maße in Mozilla Firefox reproduzierbar. Die JavaScript-Konsole und die Entwicklertools des Browsers wurden zur Beobachtung von Laufzeitverhalten, Speicheradressen und Fehlermeldungen eingesetzt.

5.2 Beispiel 1: Stack-Buffer-Overflow und Authentifizierungsumgehung

5.2.1 Implementierung

Das erste PoC-Modul (`buffer_overflow/vuln.c`) simuliert eine WASM-seitige Authentifizierungslogik. Die Zustandsstruktur `AppState` legt die Felder `username[16]`, `password[16]`, `isAdmin` und `secret` sequenziell im linearen Speicher ab. Die Funktion `attempt_login()` kopiert das übergebene Passwort mittels `strcpy()` in den 16 Byte großen Puffer, ohne die Länge der Eingabe zu prüfen:

```
strcpy(state.password, pass);
```

Das Flag `isAdmin` liegt im Memory-Layout unmittelbar hinter `password`. Ein Passwort, das länger als 15 Zeichen ist, überschreibt damit direkt das Authentifizierungsattribut.

5.2.2 Durchführung

Nach dem Start des Docker-Containers und dem Aufruf von `http://127.0.0.1:8080` im Browser wird ein Loginformular präsentiert. Bei Eingabe des korrekten Passworts (`password`) wird das geschützte Geheimnis korrekt zurückgegeben. Bei Eingabe eines Passworts mit mehr als 16 Zeichen überschreitet der `strcpy()`-Aufruf die Buffer-Grenze und setzt `isAdmin` auf einen Wert ungleich null. Die Funktion `read_secret()` gibt daraufhin den geschützten Inhalt zurück, obwohl kein gültiges Passwort eingegeben wurde.

5.2.3 Ergebnisse

Der Overflow bleibt vollständig innerhalb des linearen WASM-Speichers. Der Host-Prozess des Browsers wird nicht beeinträchtigt, und es findet kein Zugriff auf den Browser-eigenen Speicher statt. Das WASM-Sandbox-Modell verhindert somit eine Eskalation auf die Host-Ebene. Innerhalb des Moduls ist jedoch eine vollständige Authentifizierungsumgehung möglich, da die Speicherstruktur keinerlei Schutzmechanismen gegen modulinterne Überläufe besitzt.

5.3 Beispiel 2: Use-After-Free und Heap-Speicherkontrolle

5.3.1 Implementierung

Das zweite PoC-Modul (`use_after_free/vuln.c`) modelliert ein Session-Management. Eine `Session`-Struktur mit den Feldern `username[16]`, `token[16]` und `is_active` wird per `malloc()` auf dem WASM-Heap alloziert. Die Funktion `destroy_session()` gibt den Speicher frei, setzt den Pointer `g_session` jedoch nicht auf `NULL`:

```
void destroy_session(void) {
    free(g_session);
    g_freed = 1;
    /* g_session wird nicht auf NULL gesetzt */
}
```

Die Funktion `read_token()` prüft lediglich, ob `g_session` nicht `NULL` ist, und dereferenziert den Zeiger anschließend unkritisch. Da der WASM-Allocator `dlmalloc` einen freigegebenen Block bei der nächsten gleichgroßen Allokation typischerweise wiederverwendet, kann eine nachfolgende Allokation mit Angreifer-kontrollierten Daten den Speicher belegen, auf den der veraltete Pointer noch verweist.

5.3.2 Durchführung

Nach dem Erzeugen einer Session wird deren Heap-Adresse angezeigt. Nach dem Aufruf von `destroy_session()` ist der Pointer weiterhin gültig, und ein UAF-Leseaufruf von `read_token()` gibt das Token aus dem freigegebenen Speicher zurück. Wird anschließend `new_allocation()` mit einem Angreifer-kontrollierten String aufgerufen, gibt der Allocator dieselbe Adresse zurück. Ein erneuter Aufruf von `read_token()` liefert nun den Angreifer-Inhalt anstelle des ursprünglichen Tokens.

5.3.3 Ergebnisse

Der PoC demonstriert zwei Eigenschaften des linearen WASM-Speichermodells: Erstens werden freigegebene Speicherbereiche nicht sofort gelöscht oder durch das Runtime-System als ungültig markiert. Zweitens gibt es innerhalb des Moduls keine Zugriffskontrolle auf Heap-Blöcke. Der Angreifer kann damit den Inhalt eines aktiven Pointers durch eine externe Allokation kontrollieren, ohne den Browser oder die Host-Umgebung zu kompromittieren.

5.4 Beispiel 3: Integer-Overflow und Umgehung einer Längenprüfung

5.4.1 Implementierung

Das dritte PoC-Modul (`integer_overflow/vuln.c`) simuliert eine Datenübermittlung an ein WASM-Backend. Die Zustandsstruktur enthält einen 32 Byte großen Puffer `data`, gefolgt vom Flag `is_admin`. Die Größenberechnung erfolgt als `uint8_t`-Ausdruck:

```
uint8_t total = item_count * ITEM_SIZE;
if (total > sizeof(g_state.data)) {
    return 0;
}
memcpy(g_state.data, payload,
       (size_t)item_count * ITEM_SIZE);
```

Bei einem `item_count` von 32 und `ITEM_SIZE` von 8 beträgt das mathematische Produkt 256. Da `uint8_t` nur Werte von 0 bis 255 aufnehmen kann, wird 256 zu 0 gekürzt. Die Prüfbedingung `0 > 32` ist falsch, sodass die Validierung als bestanden gilt. Der anschließende `memcpy()`-Aufruf verwendet jedoch den originalen, nicht gekürzten Wert als `size_t`, was zu einem Schreiben von 256 Bytes in den 32-Byte-Puffer führt.

5.4.2 Durchführung

Die Demo-Seite ermöglicht die Eingabe eines beliebigen `item_count`-Wertes (0–255). Bei `item_count = 5` beträgt das Produkt 40 und wird korrekt abgelehnt. Bei `item_count = 32` liefert die Größenberechnung den Wert 0, die Prüfung wird umgangen, und `memcpy()` schreibt 256 Bytes. Das Flag `is_admin` an Offset 32 wird auf `0x01` gesetzt, woraufhin `read_secret()` das geschützte Geheimnis zurückgibt.

5.4.3 Ergebnisse

Dieser PoC verdeutlicht, dass ganzzahlige Überläufe in C bei der Konvertierung in schmale Typen keine Ausnahmen auslösen und vom Compiler standardmäßig nicht als Fehler behandelt werden. Die WASM-Runtime führt keine eigenständige Validierung arithmetischer Ausdrücke durch; die Korrektheit der Größenberechnung liegt vollständig in der Verantwortung des Source-Codes.

5.5 Beispiel 4: JS-WASM-Grenzvertrauen in Rust

5.5.1 Implementierung

Das vierte PoC-Modul (`rust_boundary/src/lib.rs`) demonstriert, dass Sprachsicherheit auf Modulebene keine Schutzwirkung an der Schnittstelle zwischen JavaScript-Host und WASM-Modul entfaltet. Das Modul ist in Rust geschrieben; Rust verhindert durch sein Ownership-Modell zur Compile-Zeit typische Speicherfehler wie UAF oder Buffer Overflows im sicheren Code. Die Funktion `process_request(ptr, len)` akzeptiert einen vom JavaScript-Host übergebenen Zeiger und eine Längenangabe:

```
pub extern "C" fn process_request(
    ptr: *const u8,
    len: usize
) -> *const u8 {
    unsafe {
        let input = slice::from_raw_parts(ptr, len);
        std::ptr::copy_nonoverlapping(
            input.as_ptr(),
            STATE.request.as_mut_ptr(),
            len
        );
        ...
    }
}
```

Die Länge `len` stammt ausschließlich vom Host und wird ohne Validierung direkt als Kopiergröße verwendet. Da JavaScript-Code vollständig auf den WASM-Linearspeicher zugreifen und dort beliebige Werte schreiben kann, ist die Längenangabe aus Sicht des Moduls nicht vertrauenswürdig.

5.5.2 Durchführung

Der JavaScript-Host schreibt ein kurzes Payload in den WASM-Speicher, übergibt jedoch eine absichtlich überhöhte Länge. Obwohl der Payload nur wenige Bytes umfasst, kopiert `copy_nonoverlapping()` entsprechend mehr Bytes aus dem Speicher, was in das benachbarte `is_admin`-Flag hineinschreibt und damit Administratorzugriff gewährt.

5.5.3 Ergebnisse

Rust verhindert speicherunsichere Operationen innerhalb des eigenen sicheren Codes zuverlässig. An der Grenze zum JavaScript-Host, innerhalb von `unsafe`-Blöcken, ist die Vertrauenswürdigkeit der Eingaben jedoch nicht automatisch gewährleistet. Das Beispiel zeigt, dass auch der Einsatz memory-sicherer Sprachen keine hinreichende Schutzmaßnahme darstellt, wenn Längen- und Zeigervalidierungen an der Host-Grenze fehlen.

5.6 Beispiel 5: AddressSanitizer als Gegenmaßnahme

5.6.1 Implementierung

Das fünfte PoC-Modul (`use_after_free_asan/`) verwendet identischen Quellcode zu Beispiel 2. Der einzige Unterschied liegt in den Compiler-Flags: Anstelle von `-O2` wird `-fsanitize=address -O1` verwendet:

```
emcc vuln.c -fsanitize=address -O1 \
-s MODULARIZE=1 ...
```

Emscriptens AddressSanitizer basiert auf LLVM ASan und instrumentiert jede Heap-Operation zur Compile-Zeit. Neben dem linearen WASM-Speicher wird ein Shadow-Memory-Bereich angelegt. Jeder `malloc()`-Aufruf markiert den entsprechenden Shadow-Bereich als zugänglich; `free()` markiert ihn als vergiftet (`0xfd`). Jeder Lese- und Schreibzugriff wird vor seiner Ausführung anhand des Shadow-Bytes geprüft.

5.6.2 Durchführung

Die Demo-Seite leitet den `printErr`-Kanal des Emscripten-Moduls in das Browser-Log um, sodass ASan-Diagnosemeldungen direkt im Browser sichtbar sind. Nach dem Erzeugen und Freigeben einer Session wird `read_token()` aufgerufen. ASan erkennt den Zugriff auf vergifteten Speicher, gibt eine `heap-use-after-free`-Meldung aus und ruft `abort()` auf. Das Token wird nicht zurückgegeben; das WASM-Modul hält an.

5.6.3 Ergebnisse

Der Vergleich der beiden UAF-Builds zeigt die Wirksamkeit von ASan als Entwicklungswerkzeug. Die Sicherheitslücke wird bereits beim ersten Aufruf des fehlerhaften Codepfades erkannt, bevor Angreifer-Daten exponiert werden. Gleichzeitig verdeutlicht das Experiment die Grenzen dieser Maßnahme: Der erhöhte Speicherbedarf durch Shadow-Memory sowie der Laufzeit-Overhead von etwa Faktor 2 machen ASan für produktive Deployments ungeeignet. ASan ist daher primär als Werkzeug für den Entwicklungs- und Testprozess zu verstehen, nicht als Laufzeitschutz.

6 Diskussion und Bewertung

6.1 Beantwortung der Forschungsfragen

6.2 Bewertung der Sicherheitsmechanismen

6.3 Grenzen der Untersuchung

6.4 Empfehlungen für Entwickler

7 Fazit und Ausblick

7.1 Zusammenfassung

7.2 Zukünftige Entwicklungen

7.3 Weiterführende Forschungsansätze

Literatur

- [Bar11] A. Barth. *The Web Origin Concept*. RFC 6454. 2011. URL: <https://www.rfc-editor.org/rfc/rfc6454> (besucht am 03.06.2026).
- [Byt] Bytecode Alliance. *WebAssembly Ecosystem and WASI*. URL: <https://bytecodealliance.org/> (besucht am 03.06.2026).
- [Cor+26] Lorenzo Corrias u. a. *An Analysis of Modern Web Security Vulnerabilities Inside WebAssembly Applications*. Submitted to ICISSP 2026. März 2026. arXiv: [2603.09426](https://arxiv.org/abs/2603.09426) [cs.CR].
- [Dra+25] Oussama Draissi u. a. “Wemby’s Web: Hunting for Memory Corruption in WebAssembly”. In: *Proc. ACM Softw. Eng.* 2.ISSTA (Juni 2025). DOI: [10.1145/3728937](https://doi.org/10.1145/3728937). URL: <https://doi.org/10.1145/3728937>.
- [ECM25] ECMA International. *ECMAScript Language Specification*. 2025. URL: <https://tc39.es/ecma262/> (besucht am 03.06.2026).
- [Eic15] Brendan Eich. *From ASM.JS to WebAssembly*. <https://brendaneich.com/2015/06/from-asm-js-to-webassembly/>. Accessed: 2026-06-03. Juni 2015.
- [Ems] Emscripten Project. *Emscripten Documentation*. URL: <https://emscripten.org/> (besucht am 03.06.2026).
- [Jun+24] Zhuang Junjie u. a. “Survey of WebAssembly Security”. In: *Journal of Computer Research and Development* 61.12 (2024), S. 3027–3053. ISSN: 1000-1239. DOI: [10.7544/issn1000-1239.202330049](https://doi.org/10.7544/issn1000-1239.202330049). URL: <https://crad.ict.ac.cn/en/article/doi/10.7544/issn1000-1239.202330049>.
- [LKP20] Daniel Lehmann, Johannes Kinder und Michael Pradel. “Everything Old is New Again: Binary Security of WebAssembly”. In: *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, S. 217–234. ISBN: 978-1-939133-17-5. URL: <https://www.usenix.org/conference/usenixsecurity20/presentation/lehmann>.
- [Moz13] Mozilla. *Asm.js*. <http://asmjs.org/>. Accessed: 2026-06-03. 2013.
- [PR25] Gaetano Perrone und Simon Pietro Romano. “WebAssembly and security: A review”. In: *Computer Science Review* 56 (2025), S. 100728. ISSN: 1574-0137. DOI: <https://doi.org/10.1016/j.cosrev.2025.100728>. URL: <https://www.sciencedirect.com/science/article/pii/S157401372500005X>.
- [Pyo] Pyodide Developers. *Pyodide: Python in WebAssembly*. URL: <https://pyodide.org/> (besucht am 03.06.2026).
- [Rus] Rust WebAssembly Working Group. *Rust and WebAssembly*. URL: <https://rustwasm.github.io/> (besucht am 03.06.2026).
- [SRG21] Quentin Stiévenart, Coen De Roover und Mohammad Ghafari. *The Security Risk of Lacking Compiler Protection in WebAssembly*. 2021. arXiv: [2111.01421](https://arxiv.org/abs/2111.01421) [cs.CR]. URL: <https://arxiv.org/abs/2111.01421>.
- [W3C17] W3C. *Launching the WebAssembly Working Group*. 3. Aug. 2017. URL: <https://www.w3.org/blog/2017/launching-the-webassembly-working-group/> (besucht am 03.06.2026).

- [W3C23] W3C. *Content Security Policy Level 3*. 2023. URL: <https://www.w3.org/TR/CSP3/> (besucht am 03.06.2026).
- [Web24] WebAssembly Community Group. *WebAssembly FAQ*. <https://github.com/WebAssembly/design/blob/main/FAQ.md>. Accessed: 2026-06-03. 2024.
- [Wor19a] World Wide Web Consortium. *WebAssembly Core Specification*. 2019. URL: <https://www.w3.org/TR/wasm-core-1/> (besucht am 03.06.2026).
- [Wor19b] World Wide Web Consortium. *World Wide Web Consortium (W3C) brings a new language to the Web as WebAssembly becomes a W3C Recommendation*. 5. Dez. 2019. URL: <https://www.w3.org/2019/12/pressrelease-wasm-rec.html.en> (besucht am 03.06.2026).
- [Yu+25] Zhaofeng Yu u. a. “Exploring and Exploiting the Resource Isolation Attack Surface of WebAssembly Containers”. In: *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, S. 1111–1128. ISBN: 978-1-939133-52-6. URL: <https://www.usenix.org/conference/usenixsecurity25/presentation/yu-zhaofeng>.

Eidesstattliche Erklärung

Ich versichere, dass ich die Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen oder anderen Quellen entnommen sind, sind als solche kenntlich gemacht. Die Abhandlung wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Göppingen, den 15.07.2026

(David Weber)